Software Requirements Specification (SRS)

System: Online Book Review & Recommendation Platform
Course Code: 20CYS401 — Secure Software Engineering
Version: 1.0 (Final)

1. Introduction

1.1 Purpose

This SRS defines functional, non-functional, and security requirements for the Online Book Review and Recommendation Platform. All requirements use stable IDs from `docs/TRACE-REGISTRY.md`.

1.2 Scope

A multi-user web application enabling book discovery, community ratings and reviews, personal reading lists, and administrator catalog and moderation management.

1.3 Roles

Role	Description
USER	Registered authenticated user
ADMINISTRATOR	Privileged user with `is_admin=True` flag
ANONYMOUS	Unauthenticated visitor

2. Functional Requirements

ID	Requirement	Actor	Priority	Acceptance Criteria
REQ-01	System allows new user registration with unique username (3–30 chars) and email. Validates format and rejects duplicates.	USER	HIGH	New user can register; duplicate username returns validation error
REQ-02	System authenticates users using bcrypt-hashed passwords with rounds=12. Sessions use HttpOnly, SameSite=Lax, with 1h expiry.	USER	CRITICAL	Login succeeds with valid creds; session cookie has HttpOnly flag
REQ-03	System enables parameterized book search by title keyword, author name, and/or genre using ORM-bound queries.	USER, ADMIN	HIGH	Search returns matching results; SQL injection payloads return 0 results safely
REQ-04	System displays book detail page with title, author, genre, ISBN, description, aggregated average rating, and community reviews.	USER, ADMIN	HIGH	Book detail page renders all fields and shows computed average rating
REQ-05	System allows one integer rating (1–5) per user per book. Subsequent ratings rejected with message.	USER	HIGH	Ratings 1–5 accepted; 0, 6, text rejected; second submission blocked
REQ-06	System allows user to write a review (5–2000 characters). Content stored safely with Jinja2 autoescaping on output.	USER	HIGH	Review created; XSS payload rendered as escaped text, not executed HTML
REQ-07	System allows user to edit their own review. Server-side object-level authorization prevents editing others' reviews.	USER	CRITICAL	Alice can edit her own review; Bob's attempt to edit Alice's review returns denial
REQ-08	System allows user to delete their own review. Server-side object-level authorization prevents deleting others' reviews.	USER	CRITICAL	Alice can delete her own review; Bob's attempt to delete Alice's returns denial
REQ-09	System provides private reading list per user with statuses: want-to-read, reading, finished.	USER	MEDIUM	Reading list visible only to owner; RBAC denies cross-user access
REQ-10	Administrator can create, update, and delete book catalog entries. All actions audit-logged.	ADMIN	HIGH	Admin can CRUD books; non-admin receives 403 on /admin/* routes
REQ-11	Administrator can hide/restore any review with a mandatory reason. Every action is audit-logged with actor, timestamp, reason.	ADMIN	HIGH	Admin can hide review with reason; review no longer visible in public listing

ID	Requirement	Actor	Priority	Acceptance Criteria
REQ-12	System performs complete server-side session invalidation on logout.	USER	HIGH	After logout, session cookie no longer grants access to protected routes

3. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	All passwords stored using bcrypt (rounds≥12); never logged or transmitted in plaintext
NFR-02	Security	CSRF token required on all state-changing HTML forms
NFR-03	Security	Security headers (X-Frame-Options: DENY, X-Content-Type-Options: nosniff, CSP) sent on all responses
NFR-04	Security	Error pages return HTTP status codes without stack traces or internal details
NFR-05	Security	All database access uses SQLAlchemy ORM parameterized queries
NFR-06	Reliability	Application health check endpoint `/health` returns 200 with DB connectivity confirmation
NFR-07	Auditability	All security-significant events are recorded to `AuditLog` table with actor, action, resource, IP, status, timestamp
NFR-08	Maintainability	Layered architecture (routes → services → repositories → models) enables unit testing without HTTP
NFR-09	Portability	Runs on Python 3.9+ with SQLite; containerized via Docker; deployable on Kubernetes
NFR-10	Availability	Application provides `/health` endpoint for liveness/readiness probe support

4. Security Requirements (CIA Mapping)

Requirement	Confidentiality	Integrity	Availability	Control
REQ-01 User Auth	High — password never exposed	High — bcrypt prevents reversal	Medium	CTRL-01
REQ-02 Sessions	High — HttpOnly, SameSite	High — server-side invalidation	Medium	CTRL-01
REQ-07 Review Edit	Low	CRITICAL — IDOR prevention	Low	CTRL-02
REQ-08 Review Delete	Low	CRITICAL — IDOR prevention	Low	CTRL-02
REQ-09 Reading List	High — private to owner	Medium	Low	CTRL-06
REQ-10 Admin Catalog	Low	High — admin-only RBAC	Low	CTRL-06
REQ-11 Moderation	Low	High — audit trail	Low	CTRL-06, CTRL-08

5. Constraints and Assumptions

- **Local SQLite** is used for examination; production would replace with PostgreSQL
- Python 3.9+ is required; tests run on Python 3.14.8 (verified)
- Docker desktop must be running locally to build/run containers
- No AWS or paid cloud resources are used or required